CYBERGUARD AI

System Architecture, End-to-End Flowcharts
and Complete Frontend Page / Backend API Mapping

Project: A Multi-Agent System for Cybersecurity Threat Detection and Correlation

Architecture blueprint for 56 planned pages across Public, Individual, Security Analyst and Organization experiences. Includes system architecture, collection flows, eight agents, database domains, API contracts, security controls and implementation sequence.

This is a target architecture proposal, not proof that every listed capability is already implemented or operational.

Document guide and product invariants

This document consolidates the agreed product structure and target backend blueprint. Endpoint names are proposed contracts. Inspect the existing E:\CyberGuard repository, migrations, routes and tests before changing code; reuse working modules rather than restructuring blindly.

Fixed product decisions

• Exactly three user categories: Individual / Normal User, Security Analyst, Organization / Company Admin. No Platform Admin or Super Admin workspace.

• Analyst verification is required before approved analyst permissions are granted. Pending, failed or incomplete applicants remain restricted.

• Organization verification is not required. The authenticated organization creator becomes the initial Company Admin.

• Employees and analysts can belong to multiple rooms. Use many-to-many memberships and room-scoped permissions, not one room_id on a user.

• Employees accept invitations before membership activation. Analysts need verified status plus explicit room/case assignment.

• Endpoint collectors can provide supported endpoint logs and endpoint network telemetry. Gmail is a separate OAuth integration. Wider network visibility may require a sensor/gateway.

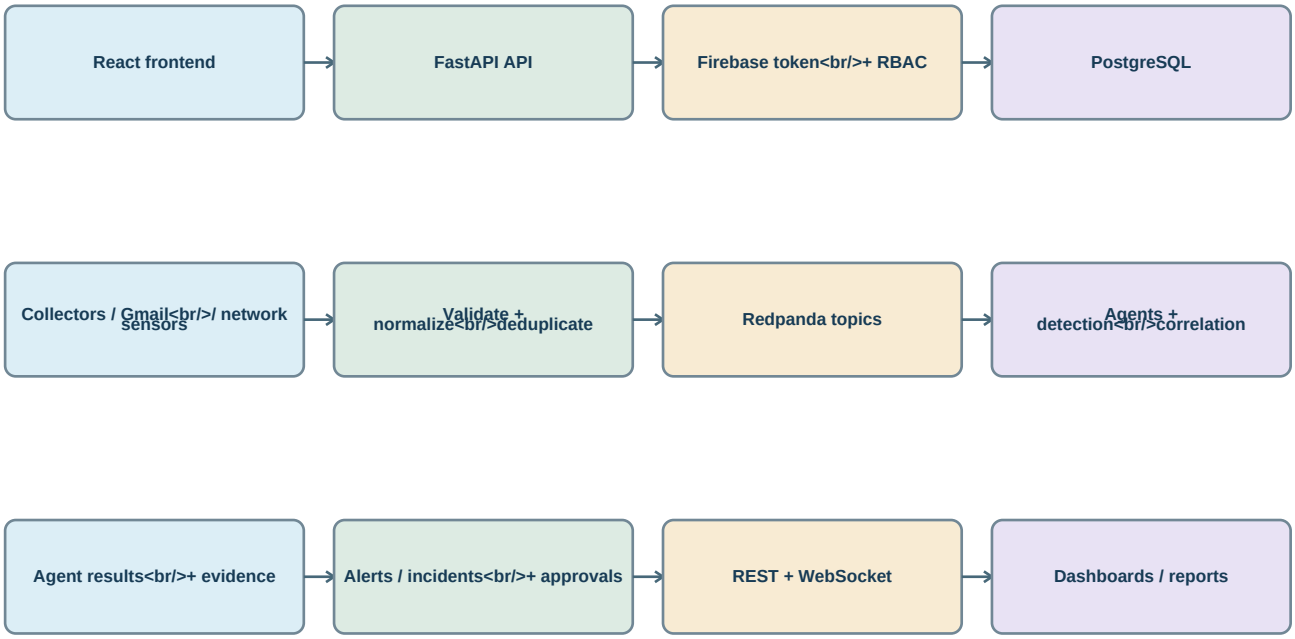
• Show zero detections, no data, stale monitoring and unavailable monitoring as distinct states. Never invent security scores or event counts.

Target technology stack

Layer	Technology / responsibility
Frontend	React 18, TypeScript, Vite, Tailwind CSS; 56 route/page experiences.
Backend	FastAPI / Python; REST APIs, validation, auth, authorization, WebSocket gateway.
Identity	Firebase Authentication; backend verifies Firebase ID tokens.
Database	PostgreSQL for users, memberships, devices, events, tasks/results, alerts, incidents, cases, reports and audit.
Streaming	Redpanda (Kafka-compatible) for durable event and task processing.
Cache	Redis for cache, rate limits, short-lived coordination and ephemeral state.
Files	Private object storage for EML, attachments, evidence and reports; database stores metadata and hashes.
Integrations	Gmail OAuth, endpoint collectors, network sensors, threat-intelligence providers, optional LLM provider.
Operations	Structured logs, metrics, traces, health/readiness checks, backups and dead-letter monitoring.

1. High-level system architecture

Separate the user-facing control plane from ingestion and asynchronous analysis. FastAPI authenticates and authorizes user actions. Collectors and integrations provide telemetry. Redpanda decouples ingestion from agent execution. PostgreSQL stores durable state; Redis supports cache/coordination; object storage holds large protected evidence files.



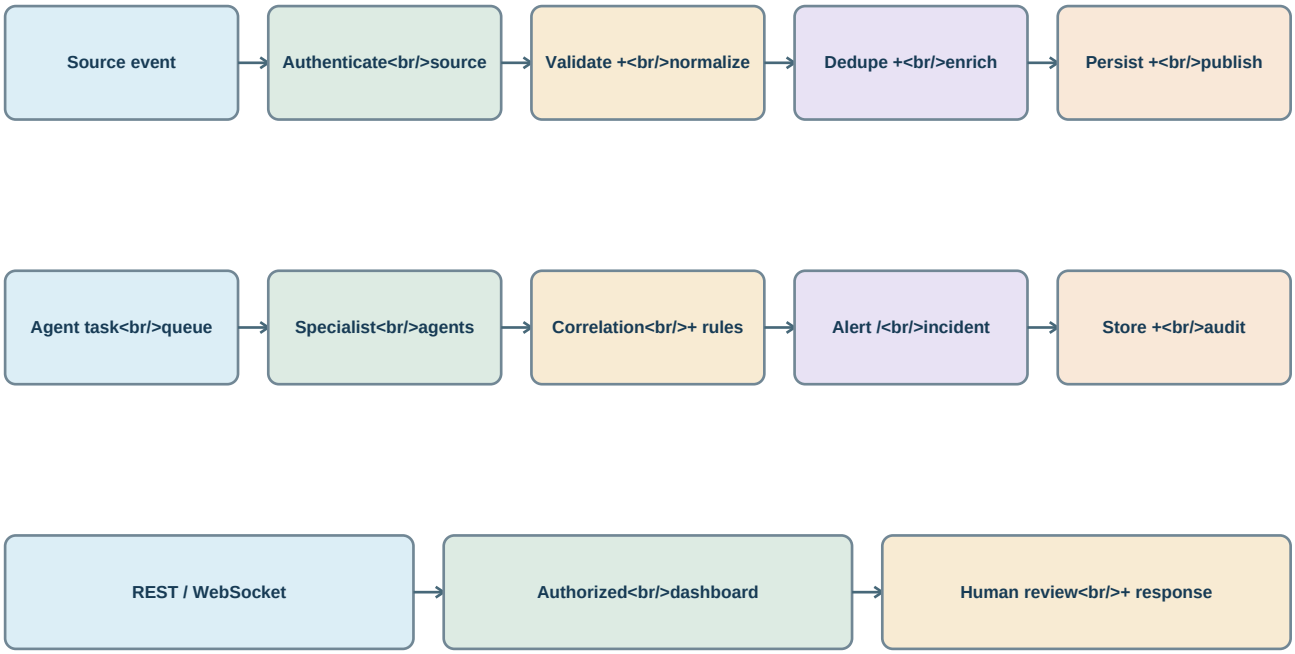
Logical component responsibilities

Component	Responsibilities	Design requirements
Frontend	Forms, dashboards, tables, filters, page routing, user action states.	Client selection never grants permission; display real backend state.
API gateway	REST, validation, token verification, pagination, rate limits.	Resolve identity, account context and resource scope on every request.
Identity / authorization	Personal workspace, org membership, analyst verification, room/case permissions.	Deny by default; backend checks; audit sensitive changes.
Ingestion	Device collector, Gmail OAuth, EML upload, network sensor, approved log sources.	Authenticate source; preserve source and ingestion timestamps.
Event pipeline	Canonical normalization, enrichment, dedupe, persistence and streaming.	Idempotency, schema versioning, replay-safe consumers.
Agent runtime	Eight specialist agents, task queue, retries, results and execution logs.	Durable task state, timeouts, versions, evidence references, DLQ.
Detection / correlation	Rules, IOC matches, entity/time correlation, severity.	Explainable rules and traceable evidence.
Incident management	Alert lifecycle, incident grouping, assignment, notes, approval.	Human approval for consequential response actions.
Realtime	WebSocket updates for authorized resources.	Authorize every subscription by user, organization, room and resource.

Control plane vs data plane

Control plane: login, account context, enrollment, Gmail connect, room administration, invitations, case assignment, approvals and report requests. **Data plane:** telemetry intake, parsing, event streaming, agent execution, enrichment, detection, correlation and alert/incident updates.

2. End-to-end event processing flow



Stage-by-stage explanation

Stage	Processing	Required record / control
1. Intake	Device, Gmail, EML, network sensor or approved log source submits data.	Source identity, owner/scope, source ID, request/job ID, ingestion time.
2. Authenticate	Verify collector credentials or OAuth connection; resolve workspace/org/room.	Reject revoked/unknown sources; prevent cross-tenant ingestion.
3. Normalize	Parse source-specific data into canonical event schema.	Schema version, source event time, ingestion time, normalized actor/host/IP.
4. Dedupe / enrich	Create stable dedupe key; attach asset/user and available threat-intel context.	Idempotency key, enrichment source/version/time, IOC references.
5. Persist / publish	Persist event metadata and publish event reference to Redpanda.	Outbox or equivalent reliability; event ID, topic and delivery state.
6. Agent execution	Schedule applicable agents and track task attempts/results.	Agent/version, attempts, timeout, evidence, confidence, severity, trace ID.
7. Detect / correlate	Apply rules and correlate related events, IOCs, entities and time windows.	Rule/version, correlation group, matched evidence and rationale.
8. Alert / incident	Create/update alert; group related alerts into incident when appropriate.	Lifecycle, owner, scope, timestamps, evidence and audit history.
9. Present	REST loads current state; WebSocket sends authorized change notifications.	Per-user/room authorization, reconnect and replay strategy.

Reliability principles

- Ingestion and consumers must be idempotent; retries must not create duplicate incidents or payments.
- Use bounded retries, exponential backoff and a dead-letter queue for poison messages.
- Keep event time distinct from ingestion time; support late-arriving events.
- Persist accepted background jobs so browser closure does not cancel analysis.
- Preserve original evidence and provenance; derived conclusions must link to source records.

3. Collection and integration flows

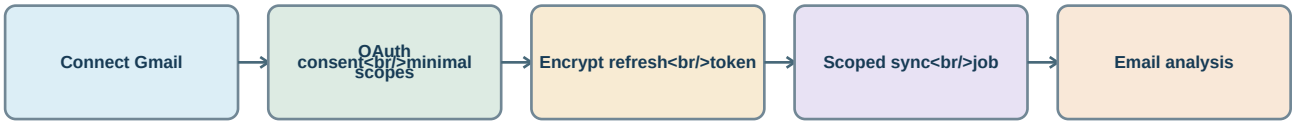
A connected endpoint does not imply mailbox access, and endpoint network metadata is not full network packet visibility.

3.1 Endpoint enrollment



Create a single-use, expiring enrollment token and store only its hash. The collector exchanges it for device credentials, registers device identity and sends authenticated heartbeats/events. Revoke credentials when disconnected or compromised. Display collector heartbeat and last successful event separately.

3.2 Gmail OAuth



Gmail is a separate integration. Disclose the exact mailbox access requested; use minimum necessary scopes; encrypt refresh tokens; never return them to the browser or log them. Allow sync configuration, disconnect and revocation. Support validated .eml upload as a separate manual path.

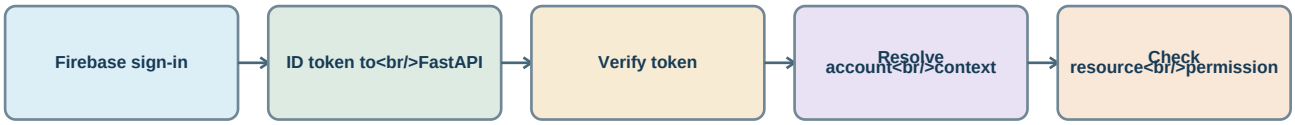
3.3 Network visibility

Endpoint network telemetry describes connections observed on that endpoint. Broader visibility may require a dedicated sensor, gateway, firewall, cloud flow logs or other explicitly integrated source. The UI must state what is monitored and from which source.

Source state model

State	Meaning / display
Not connected	No integration exists; show Connect source.
Enrolling / authorizing	Setup initiated but not validated; show Setup in progress.
Connected, no events	Credentials valid but no data received; show Waiting for data.
Healthy / reporting	Heartbeat or sync is current and events arrive; show last event time.
Degraded	Partial errors, delayed sync, parse failures or stale heartbeat; explain cause.
Offline / expired	Heartbeat or OAuth credentials no longer valid/current; offer reconnect.
Revoked / disconnected	Access explicitly disabled; show Disconnected.

4. Authentication and authorization



Firebase authenticates identity; it does not authorize application roles. A login account selector is a context hint only. Backend must independently resolve account membership, analyst verification and resource scope.

Authorization invariants

• A user may have multiple account contexts; selected type never grants permissions.

• Individual access is scoped to personal workspace and authorized devices/integrations.

• Company Admin access is limited to organizations where the user has active admin membership.

• Employee access is limited to active organization/room membership and granted permissions.

• Analyst access requires verified status plus explicit assignment to case, room or resource.

• Every request checks identity, context, workspace/org, room, ownership, membership and action permission.

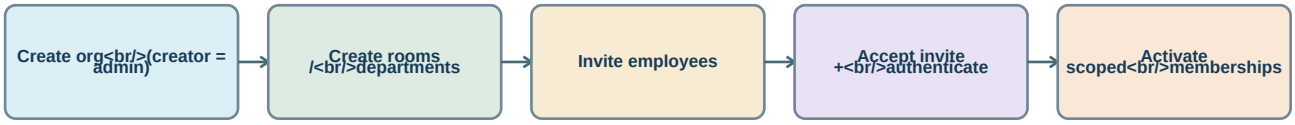
• Hiding UI controls is not security. Enforce permissions on every backend route and query.

Analyst verification flow



Pending, retry, needs-more-information or rejected applications must not receive approved analyst permissions. Verification and customer/room assignment are separate checks.

Organization / room lifecycle



No company verification gate. Admin can create/archive rooms and invite employees. Employees may belong to multiple rooms; approved analysts may be assigned to multiple rooms. Organization dashboards aggregate only rooms the admin may administer. Preserve historical records when rooms are archived.

5. Eight-agent execution model

Agent	Responsibility	Inputs / outputs
1 Email Analysis	Analyze sender, headers, URLs, attachments and content.	Gmail message or EML; structured risk, indicators, reasons and evidence.
2 Log Analysis	Analyze Windows, Linux/SSH, web app and database logs.	Normalized events; anomalies, suspicious sequences, identities/hosts and evidence.
3 Network Analysis	Analyze endpoint connections and sensor/flow telemetry.	Connection metadata; suspicious connections, scan/beacon indicators and evidence.
4 AI Analyst	Evidence-grounded investigation assistant.	Authorized event/incident/case context; cited summary and clearly marked hypotheses.
5 Threat Intelligence	Enrich IPs, domains, URLs, hashes and supported indicators.	IOC values; provider matches, source, timestamp and freshness.
6 IOC Correlation	Correlate indicators/entities across events and time windows.	Events and IOC matches; correlation groups and rationale.
7 Incident Response	Recommend or execute approved response workflows.	Incident, playbook, policy and approvals; proposed action and outcome.
8 Reporting	Generate reports from persisted authorized evidence.	Date range, filters and evidence refs; report job and file.

Common task envelope

Every task should include task_id, tenant/workspace scope, optional room_id, source_event_ids, target_resource_id, agent_name/version, schema_version, requested_by, created_at, trace_id, idempotency_key, priority and deadline. Results should include status, timestamps, findings, evidence_refs, severity, confidence, error_code, retryable and agent_version.

Agent safety and quality

- Agents access only data authorized for the task scope.
- Distinguish observed fact, deterministic rule match, external reputation and LLM hypothesis.
- Record model/provider/version, policy version, execution timestamps, evidence references and errors.
- Do not label confidence as probability unless calibrated and documented.
- Consequential response actions require policy checks and human approval unless explicitly pre-approved.

6. Detection, correlation, alerts and incidents

Object	Purpose / lifecycle fields
Security Event	Normalized observation: event_id, source_id, scope, event_type, event_time, ingested_at, severity, payload_ref, dedupe_key.
Agent Result	Interpretation: agent/version, status, findings, evidence_refs, confidence, timestamps, trace_id.
IOC Match	Indicator/provider match: type/value, status, reputation, observed_at, expires_at.
Correlation Group	Related events/entities/IOCs: rule/version, member refs, time window, rationale.
Alert	Actionable detection: title, severity, status, scope, first/last seen, evidence, assignee.
Incident	Investigation container: status, priority, scope, owner, timeline, notes, response actions, closure reason.

Suggested state transitions

Resource	States / rules
Alert	New → Acknowledged → In Progress → Resolved or Dismissed. Dismissal requires reason and audit.
Incident	Open → Investigating → Contained → Resolved → Closed. Do not auto-close only because detection stopped.
Agent task	Queued → Running → Succeeded / Failed / Timed Out / Cancelled. Track attempts.
Report job	Queued → Running → Completed / Failed / Expired. Protect generated files.

A recommendation is not an executed action. Show proposed, awaiting approval, approved, executing, succeeded, failed or rolled back. Approval records should include approver, policy, parameters, timestamp, expiry and rationale.

7. Database domain model and relationship guidance

Conceptual domain inventory only; final columns, indexes, constraints, retention and partitioning are to be designed before migrations.

Domain	Proposed entities	Relationship rules
Identity	users, user_account_types, user_sessions, user_preferences, legal_acceptances	One identity may have multiple account contexts; account type is not privilege escalation.
Individual	personal_workspaces, personal_workspace_memberships	Workspace ownership and optional collaboration scope.
Organization	organizations, organization_memberships, rooms, room_memberships, teams, team_memberships, invitations, room_permissions	Many-to-many membership; organization and room scope explicit.
Analyst	analyst_applications, analyst_verifications, analyst_credentials, analyst_services	Verification separate from case/room assignment.
Devices	devices, device_enrollments, device_credentials, collector_heartbeats, collector_versions, device_room_assignments	Device is personal OR org-owned; room assignment explicit.
Sources	data_sources, data_source_connections, gmail_connections, data_source_sync_jobs, network_sensors	Encrypted secrets; connection owner/scope explicit.
Email	email_messages, email_attachments, email_analyses, email_analysis_evidence	Protected content references; minimize retained content.
Events	security_events, event_sources, event_deduplication, event_payloads	Canonical event has source and scope; large raw payloads in private storage.
Agents	agent_tasks, agent_task_attempts, agent_results, agent_execution_logs	Version, attempt, provenance, status, errors and trace IDs.
Threat intel	ioc_records, threat_intelligence_sources, threat_intelligence_matches	Provider, query time, freshness and normalized indicator.
Correlation	correlation_groups, correlation_members, detection_rules, detection_rule_versions	Join tables for correlated objects; version rules.
Incidents	alerts, alert_evidence, incidents, incident_alerts, incident_evidence, incident_assignments, incident_notes	Keep alert/incident/evidence relationships explicit and auditable.
Response	response_actions, approval_records, playbook_runs	Separate proposal, approval, execution and outcome.
Cases / evidence	cases, case_assignments, case_evidence, evidence_objects, evidence_access_logs	Analyst assignment and evidence access scoped and audited.
Communication	messages, message_threads, message_attachments	Authorize thread participants and resource scope.
Finance / reports	payments, invoices, subscriptions, reports, report_jobs	Provider references only; no raw card data; private report files.
Audit	audit_logs	Append-only or tamper-evident history for sensitive actions.

Database design rules

• Use UUID keys, UTC timestamps, foreign keys, uniqueness constraints and explicit actor fields.

• Do not store one room_id on a user; use room_memberships.

• Model device ownership as personal workspace or organization, not ambiguous dual ownership.

• Every event has resolved scope, source provenance, event time and ingestion time.

• Use join tables for many-to-many event/alert/incident/evidence/case/correlation relationships.

• Store large payloads in private object storage; DB stores key, hash, size, MIME type and access metadata.

• RLS may provide defense in depth, but application authorization remains mandatory.

8. API conventions and reusable contracts

Method	Purpose	Example
GET	Read/list/search with pagination and filters.	GET /api/v1/personal/devices
POST	Create, submit analysis, initiate job or explicit action.	POST /api/v1/personal/email/analyses
PATCH	Partial update.	PATCH /api/v1/organization/rooms/{room_id}
DELETE	Rare safe deletion; prefer archive/revoke.	DELETE /api/v1/...
POST action	Explicit state transition with validation and audit.	POST /api/v1/alerts/{id}/acknowledge

Response contracts

Resource: return stable ID, state and timestamps. **List:** items, pagination metadata and applied filters. **Async job:** HTTP 202 with job_id, status, status_url, accepted_at and trace ID. Frontend polls or subscribes and shows queued/running/completed/failed.

Error contract

Status	Meaning	UI response
400	Malformed request / invalid transition	Show form or action error.
401	Missing/invalid/expired auth	Re-authenticate.
403	Authenticated but unauthorized	Show access denied; do not leak details.
404	Not found or concealed	Show unavailable/not found.
409	Conflict / duplicate / stale version	Refresh and explain conflict.
422	Schema/business validation	Map errors to fields.
429	Rate limit/provider quota	Respect Retry-After.
5xx	Unexpected failure	Show safe request ID; log trace securely.

API security

• Verify Firebase ID tokens on protected requests; never trust browser-supplied role, user, organization, room or analyst status.

• Use explicit authorization dependencies for workspace, organization, room, case, incident and evidence.

• Protect against IDOR; check resource scope in database queries as well as route dependencies.

• Rate-limit enrollment, uploads, OAuth callbacks, analysis and AI requests.

• Use narrow CORS, secure headers, CSRF protection where cookie sessions are used.

• Never log passwords, tokens, API keys, raw sensitive email content or full evidence payloads.

9. Complete page-to-backend API mapping

The following 56-page map uses representative proposed endpoints. Align route names with the existing FastAPI router. Each page action must have loading, success, error, permission and audit behavior.

Page / route	Backend responsibility	Representative APIs	Key actions
1 Home /	Public config/status	GET /api/v1/public/config; GET /api/v1/public/status	CTA → account selection.
2 User Selection /select-account	Current legal versions	GET /api/v1/legal/current-versions	Select one of three account types; continue.
3 Login /login	Resolve account contexts	GET /api/v1/auth/me; GET /api/v1/auth/account-contexts; POST /api/v1/auth/session/validate	Firebase email/Google sign-in; forgot password; sign out.
4 Individual Register /register?type=individual	Profile, personal workspace, legal acceptance	POST /api/v1/users/profile; POST /api/v1/personal/workspaces; POST /api/v1/legal/acceptances	Register; verify email; create workspace idempotently.
5 Personal Dashboard /personal/dashboard	Workspace metrics, activity, notifications	GET /api/v1/personal/dashboard; GET /api/v1/personal/activity	Refresh, date filter, quick actions, live updates.

Page / route	Backend responsibility	Representative APIs	Key actions
6 My Devices /personal/devices	Owned devices, enrollment and health	GET /api/v1/personal/devices; POST /api/v1/personal/devices/enrollments; GET /api/v1/personal/devices/enrollments/{id}	Connect, copy instructions, revoke, refresh.
7 Device Details /personal/devices/{deviceId}	Metadata, heartbeat, events, diagnostics	GET /api/v1/personal/devices/{id}; GET /api/v1/personal/devices/{id}/health; GET /api/v1/personal/devices/{id}/events	Rename, inspect, revoke, download diagnostics.
8 Email Security /personal/email	Gmail connection, sync, EML upload, analyses	GET /api/v1/personal/email/connections; POST /api/v1/integrations/gmail/connect; POST /api/v1/personal/email/sync; POST /api/v1/personal/email/uploads	Connect/disconnect Gmail, sync, upload EML, analyze.
9 Email Analysis Details /personal/email/{analysisId}	Status, indicators, evidence	GET /api/v1/personal/email/analyses/{id}; GET /api/v1/personal/email/analyses/{id}/evidence; POST /api/v1/personal/email/analyses/{id}/reanalyze	Reanalyze, export, feedback.
10 Log Monitoring /personal/logs	Sources, normalized events, log results	GET /api/v1/personal/logs/sources; GET /api/v1/personal/logs/events; POST /api/v1/personal/logs/search	Filter source/time/severity; connect source; investigate.

Page / route	Backend responsibility	Representative APIs	Key actions
11 Network Security /personal/network	Endpoint network telemetry and sensors	GET /api/v1/personal/network/overview; GET /api/v1/personal/network/connections; GET /api/v1/personal/network/sources	Connect sensor; filter; investigate indicator.
12 Threat Alerts /personal/alerts	Personal alerts and lifecycle	GET /api/v1/personal/alerts; POST /api/v1/alerts/{id}/acknowledge; POST /api/v1/alerts/{id}/dismiss	Search, filter, acknowledge, dismiss with reason.
13 Alert Details /personal/alerts/{alertId}	Evidence, findings, correlation and timeline	GET /api/v1/personal/alerts/{id}; GET /api/v1/alerts/{id}/evidence; GET /api/v1/alerts/{id}/timeline	Acknowledge, link/create incident, add note.
14 Incidents /personal/incidents	List/create personal incidents	GET /api/v1/personal/incidents; POST /api/v1/personal/incidents	Create, filter, export, open.
15 Incident Details /personal/incidents/{incidentId}	Timeline, alerts, evidence, notes, approvals	GET /api/v1/personal/incidents/{id}; POST /api/v1/incidents/{id}/notes; POST /api/v1/incidents/{id}/actions	Assign, note, request/approve action, close with reason.

Page / route	Backend responsibility	Representative APIs	Key actions
16 AI Security Assistant /personal/ai-assistant	Evidence-scoped conversation	POST /api/v1/personal/ai/sessions; POST /api/v1/personal/ai/sessions/{id}/messages	Ask, cancel, feedback; cite authorized evidence.
17 Security Analysts /personal/analysts	Browse verified public analyst profiles	GET /api/v1/personal/analysts; GET /api/v1/personal/analysts/{id}; POST /api/v1/personal/service-requests	Search, profile, request service.
18 My Services /personal/services	Service requests and deliverables	GET /api/v1/personal/service-requests; GET /api/v1/personal/service-requests/{id}	View status, message, cancel if allowed.
19 Reports /personal/reports	Personal reports and downloads	GET /api/v1/personal/reports; POST /api/v1/personal/reports; GET /api/v1/reports/{id}/download	Generate, filter, download.
20 Settings /personal/settings	Profile, preferences, sessions	GET /api/v1/users/me; PATCH /api/v1/users/me; GET /api/v1/users/me/sessions	Update, revoke sessions, disconnect integrations.

Page / route	Backend responsibility	Representative APIs	Key actions
21 Analyst Registration /analyst/register	Applicant profile and legal acceptance	POST /api/v1/analyst/applications; POST /api/v1/legal/acceptances	Submit application; no analyst permissions yet.
22 Application /analyst/application	Qualifications and supporting details	GET /api/v1/analyst/applications/me; PATCH /api/v1/analyst/applications/me; POST /api/v1/analyst/applications/me/submit	Save draft, upload, submit.
23 Verification /analyst/verification	Identity/credential checks	POST /api/v1/analyst/verifications; GET /api/v1/analyst/verifications/{id}	Start check, provide evidence, retry permitted checks.
24 Application Status /analyst/application-status	Pending/approved/more-info/retry/rejected state	GET /api/v1/analyst/applications/me/status	Refresh, provide requested information.
25 Analyst Dashboard /analyst/dashboard	Assigned case and room workload	GET /api/v1/analyst/dashboard; GET /api/v1/analyst/notifications	Refresh, date filter, open assigned work.

Page / route	Backend responsibility	Representative APIs	Key actions
26 My Cases /analyst/cases	Cases explicitly assigned to verified analyst	GET /api/v1/analyst/cases	Filter, open, update permitted status.
27 Available Cases /analyst/available-cases	Eligible case summaries	GET /api/v1/analyst/available-cases; POST /api/v1/analyst/available-cases/{id}/apply	View summary, apply.
28 Case Details /analyst/cases/{caseId}	Case, assignment, timeline, findings	GET /api/v1/analyst/cases/{id}; PATCH /api/v1/analyst/cases/{id}; POST /api/v1/analyst/cases/{id}/notes	Update permitted fields, add findings.
29 Evidence /analyst/evidence	Evidence assigned-case index and access logs	GET /api/v1/analyst/evidence; GET /api/v1/analyst/evidence/{id}	View/download if authorized; record access.
30 AI Investigation /analyst/ai-investigation	Case-scoped AI assistance	POST /api/v1/analyst/ai-investigations; POST /api/v1/analyst/ai-investigations/{id}/messages	Ask, summarize, cite evidence.

Page / route	Backend responsibility	Representative APIs	Key actions
31 Messages /analyst/messages	Authorized service/case communication	GET /api/v1/analyst/message-threads; POST /api/v1/messages	Send, attach, mark read.
32 Reports /analyst/reports	Case reports and review workflow	GET /api/v1/analyst/reports; POST /api/v1/analyst/reports; POST /api/v1/analyst/reports/{id}/submit	Draft, submit, revise, download.
33 My Services /analyst/services	Service catalog and assigned requests	GET /api/v1/analyst/services; GET /api/v1/analyst/service-requests	Manage offering or request if permitted.
34 Profile & Verification /analyst/profile	Profile and verification state	GET /api/v1/analyst/profile; PATCH /api/v1/analyst/profile; GET /api/v1/analyst/verification-status	Edit profile, update evidence.
35 Earnings & Payments /analyst/earnings	Earnings ledger, payout, invoices	GET /api/v1/analyst/earnings; GET /api/v1/analyst/payouts; GET /api/v1/analyst/invoices	View history; provider payout flow.

Page / route	Backend responsibility	Representative APIs	Key actions
36 Analyst Settings /analyst/settings	Preferences and sessions	GET /api/v1/analyst/settings; PATCH /api/v1/analyst/settings	Update preferences, revoke sessions.
37 Organization Creation /organization/create	Create org; creator initial admin	POST /api/v1/organizations	Create, validate name/slug.
38 Organization Onboarding /organization/onboarding	Configuration only; no verification gate	GET /api/v1/organizations/{orgId}/onboarding; PATCH /api/v1/organizations/{orgId}/onboarding	Save setup, create rooms, invite team.
39 Organization Dashboard /organization/dashboard	Aggregate authorized room metrics	GET /api/v1/organizations/{orgId}/dashboard; GET /api/v1/organizations/{orgId}/activity	Filter by room/time; live updates.
40 Rooms / Departments /organization/rooms	Room lifecycle and memberships	GET /api/v1/organizations/{orgId}/rooms; POST /api/v1/organizations/{orgId}/rooms; PATCH /api/v1/rooms/{roomId}; POST /api/v1/rooms/{roomId}/archive	Create, edit, archive, assign members.

Page / route	Backend responsibility	Representative APIs	Key actions
41 Room Workspace /organization/rooms/{roomId}	Room dashboard, members, devices, sources, events	GET /api/v1/rooms/{roomId}; GET /api/v1/rooms/{roomId}/dashboard; GET /api/v1/rooms/{roomId}/members	Manage members and room resources.
42 Employees /organization/employees	Directory, invites and room assignment	GET /api/v1/organizations/{orgId}/employees; POST /api/v1/organizations/{orgId}/invitations; POST /api/v1/invitations/{id}/resend; POST /api/v1/rooms/{roomId}/members	Invite, resend, deactivate, assign multiple rooms.
43 Teams /organization/teams	Teams and team memberships	GET /api/v1/organizations/{orgId}/teams; POST /api/v1/organizations/{orgId}/teams; PATCH /api/v1/teams/{id}	Create, edit, add/remove members.
44 Devices /organization/devices	Org devices and room assignment	GET /api/v1/organizations/{orgId}/devices; POST /api/v1/organizations/{orgId}/devices/enrollments; POST /api/v1/devices/{id}/assign-room	Enroll, assign/reassign, revoke.
45 Device Details /organization/devices/{deviceId}	Health, telemetry, diagnostics, room scope	GET /api/v1/organization/devices/{id}; GET /api/v1/devices/{id}/health; GET /api/v1/devices/{id}/events	Inspect, rename, reassign, revoke.

Page / route	Backend responsibility	Representative APIs	Key actions
46 Data Sources /organization/sources	Log, network and supported integrations	GET /api/v1/organizations/{orgId}/sources; POST /api/v1/organizations/{orgId}/sources; POST /api/v1/sources/{id}/sync	Connect, configure, sync, test, disconnect.
47 Security Events /organization/events	Search events in authorized room scope	GET /api/v1/organizations/{orgId}/events; POST /api/v1/organizations/{orgId}/events/search	Filter, export, inspect evidence.
48 Threat Alerts /organization/alerts	Org alerts and lifecycle	GET /api/v1/organizations/{orgId}/alerts; POST /api/v1/alerts/{id}/acknowledge; POST /api/v1/alerts/{id}/assign	Filter, acknowledge, assign, dismiss.
49 Incidents /organization/incidents	Org incidents across authorized rooms	GET /api/v1/organizations/{orgId}/incidents; POST /api/v1/organizations/{orgId}/incidents	Create, search, filter, assign.
50 Incident Details /organization/incidents/{incidentId}	Timeline, alerts, evidence, approvals	GET /api/v1/organization/incidents/{id}; POST /api/v1/incidents/{id}/notes; POST /api/v1/incidents/{id}/approvals	Assign analyst, approve/reject, resolve.

Page / route	Backend responsibility	Representative APIs	Key actions
51 Analysts /organization/analysts	Verified analyst search and room assignments	GET /api/v1/organizations/{orgId}/analysts; POST /api/v1/rooms/{roomId}/analysts	Assign verified analyst; remove scoped assignment.
52 Communication /organization/messages	Organization/room communication	GET /api/v1/organizations/{orgId}/message-threads; POST /api/v1/messages	Send, attach, mark read.
53 Reports /organization/reports	Organization/room reports	GET /api/v1/organizations/{orgId}/reports; POST /api/v1/organizations/{orgId}/reports; GET /api/v1/reports/{id}/download	Generate, schedule if supported, download.
54 Billing /organization/billing	Subscription, invoices and provider references	GET /api/v1/organizations/{orgId}/billing; GET /api/v1/organizations/{orgId}/invoices	View plan, invoices, billing portal.
55 Organization Settings /organization/settings	Profile, policies, notifications, memberships	GET /api/v1/organizations/{orgId}/settings; PATCH /api/v1/organizations/{orgId}/settings	Update settings, manage admins/policies.

Page / route	Backend responsibility	Representative APIs	Key actions
56 Audit Logs /organization/audit-logs	Organization audit history	GET /api/v1/organizations/{orgId}/audit-logs	Filter/export; audit history cannot be edited.

10. Shared APIs, real-time and job contracts

API family	Representative endpoints	Used by
Auth	/api/v1/auth/me; /api/v1/auth/account-contexts	All protected pages.
Profile / sessions	/api/v1/users/me; /api/v1/users/me/sessions	Individual, analyst, org.
Legal	/api/v1/legal/current-versions; /api/v1/legal/acceptances	Registration and compliance.
Files	/api/v1/files/uploads; /api/v1/files/{id}/download	EML, evidence, reports, attachments.
Notifications	/api/v1/notifications; /api/v1/notifications/{id}/read	Dashboards and alerts.
Health	/api/v1/health; /api/v1/ready; restricted /metrics	Operations.

WebSocket event examples

Event	Purpose
device.status_changed	Update authorized device status.
security_event.ingested	Notify event views without broadcasting raw sensitive payload.
agent_task.updated	Update analysis progress.
alert.created / alert.updated	Update alerts and notification counters.
incident.updated	Refresh incident timeline/status.
report_job.updated	Show report generation progress.

Every WebSocket subscription is authorized by user and resource scope. A message envelope should contain event_id, event_type, emitted_at, resource_type/id, organization_id/room_id when applicable, payload_version and minimal payload. Long-running operations return HTTP 202 with job_id/status/status_url; state is persisted and can be polled or streamed.

11. Button-to-backend behavior

Action	Backend behavior / required UX
Create	POST; validate and authorize; return stable resource ID; prevent duplicates.
Edit / Save	PATCH; validate changed fields; handle stale-version conflicts.
Archive	Explicit archive action; preserve historical records and relationships.
Delete	Rare; only safely deletable records. Prefer revoke/archive.
Search / Filter	GET or POST search; apply scope before results.
Refresh	Fetch current server state and show last-updated time.
Connect	Create enrollment/OAuth flow; do not show connected before verification.
Disconnect / Revoke	Invalidate credentials/tokens; record actor, time and reason.
Sync / Analyze	Create durable job; return job ID; show progress and failure state.
Acknowledge / Resolve	Explicit lifecycle transition; record actor/time/reason.
Assign / Invite	Check eligibility and scope; invitation acceptance activates membership.
Approve / Reject	Check reviewer authority; record rationale and decision history.
Ask AI	Retrieve only authorized evidence; cite sources; distinguish facts from hypotheses.
Download / Export	Re-check access at download time; use expiring private links.
Sign out	Firebase signOut; revoke backend session where supported.

Action states: Idle → Submitting → Accepted/Processing → Succeeded or Failed. Disable duplicate submits, retain form input on validation errors, retry only retryable failures, and require confirmation for irreversible actions.

12. Security and production readiness

Area	Minimum target requirements
Authentication	Verify Firebase tokens; secure sessions, sign-out and revocation.
Authorization	Deny by default; tenant/workspace/room/case checks; IDOR and isolation tests.
Secrets	Secret manager or environment injection; encryption, rotation; never commit .env.
OAuth	Minimum Gmail scopes, encrypted refresh tokens, callback state validation, revocation.
Device trust	Single-use expiring enrollment token, hashed token, authenticated credentials, revocation, signed collector releases.
Uploads	Size/type limits, safe parsing, private storage, content hashes and access logs.
Database	Backups, migrations, least-privilege DB user, TLS, indexes, restore tests.
Streaming	Producer/consumer auth, topic ACLs, schema validation, replay safety, DLQ.
AI / LLM	Authorized retrieval only; defend against prompt injection in emails/logs; cite evidence; advisory outputs.
Response	Policy checks, human approval for impactful actions, idempotency and rollback plan.
Observability	Structured logs, metrics, traces, health checks, queue lag and ingestion gap alerts.
Resilience	Timeouts, bounded retries, circuit breakers, graceful degradation and disaster recovery.
Testing	Unit, integration, contract, authorization, tenant isolation, replay, load and failure tests.

Operational metrics

• Ingestion: accepted/rejected events, parse failures, source lag, event-time delay and duplicate rate.

• Collectors: active devices, heartbeat age, last-event age, version distribution and enrollment failures.

• Streaming: throughput, consumer lag, retries, dead-letter count and oldest unprocessed message.

• Agents: queue wait, execution duration, success/failure/timeout, provider quota and evidence completeness.

• Detection: alert counts by severity/source/rule, false-positive feedback and time-to-detect.

• Platform: API latency/errors, DB pool, cache, WebSocket connections and report failures.

Dashboard truthfulness

0 detected threats: monitoring healthy, no threats detected. **No data:** no events received. **Monitoring unavailable:** source disconnected/stale. **Analysis pending:** job incomplete. Never show a green security score unless formula, coverage, freshness and limitations are documented.

13. Recommended implementation order

Phase	Scope	Exit criteria
00	Repository audit, config and baseline	Inventory current routes/models/tests; secrets protected.
01	Identity, Firebase verification, account contexts, legal acceptance	Login/register; authorization tests.
02	Individual workspace and profile	Idempotent workspace creation and personal scope.
03	Device enrollment / collector lifecycle	Enrollment, device auth, heartbeat, revoke, ingestion.
04	Gmail OAuth and email intake	Consent, token encryption, sync jobs, EML upload.
05	Log ingestion and normalization	Windows/Linux/web/database adapters and dedupe tests.
06	Network telemetry and sensors	Endpoint and wider sensor paths.
07	Detection, IOC, alerts and incidents	Evidence-backed rules and lifecycle.
08	AI assistant	Scoped retrieval, safe prompt handling, evaluation.
09	Analyst application and verification	Pending/approved/retry/rejected; no premature access.
10	Cases, evidence and assignment	Case authorization and evidence audit.
11	Services, messages and payments	Scoped communication; payment provider; no raw card data.
12	Organizations, rooms, memberships and invites	Creator admin; many-to-many memberships.
13	Organization collection	Room-scoped devices and data sources.
14	Org incidents, analysts and audit	Admin/room scope; verified analyst assignment.
15	Reports and exports	Async jobs and private file access.
16	Settings and deployment hardening	Secrets, backups, monitoring, security/load/restore tests.

Implementation discipline

- Inspect existing files and migrations before each phase; do not overwrite working modules.
- Build one vertical slice: database → schema → service → API → authorization tests → frontend.
- Test success, validation, unauthenticated, unauthorized, cross-tenant, retry and partial failure.
- Do not mark a phase complete until acceptance criteria and automated tests pass.

14. Proposed backend module structure

Logical proposal only; adapt to existing repository rather than forcing a rewrite.

```
backend/ app/ main.py core/ # settings, security, logging, exceptions api/v1/ # routers and dependencies auth/ #
Firebase verification, account contexts authorization/ # workspace/org/room/case permissions models/ # SQLAlchemy models
schemas/ # Pydantic request/response schemas repositories/ # database access services/ # business workflows collectors/
# enrollment and ingestion integrations/ gmail/ threat_intel/ storage/ payments/ events/ # canonical schema, validation,
dedupe messaging/ # Redpanda producers/consumers agents/ # eight agent implementations detection/ # rules and detection
engine correlation/ # IOC/entity/time correlation incidents/ # alerts, incidents, response reporting/ # report jobs and
templates realtime/ # WebSocket auth/subscriptions tasks/ # background jobs observability/ # metrics, traces, health
alembic/ tests/ unit/ integration/ authorization/ contract/ end_to_end/ pyproject.toml requirements.txt alembic.ini
```

Deployment guidance

A modular monolith plus separate worker processes can be a practical initial deployment while preserving domain boundaries. Deploy API, workers/agent consumers, PostgreSQL, Redpanda, Redis, private object storage and observability with health checks and resource limits. Keep databases, brokers and internal metrics off the public internet. Separate development, staging and production secrets and backups.

15. Acceptance checklist and next step

- 56 planned pages: 4 public, 16 individual, 16 analyst and 20 organization.
- Exactly three account categories; no platform-admin workspace.
- Organization creator becomes initial Company Admin; no organization verification.
- Analyst verification gates permissions; verification does not grant access to every customer room.
- Employees and analysts can belong to multiple rooms through many-to-many membership.
- Devices, sources, events, alerts, incidents, cases, evidence and reports have explicit scope.
- Endpoint telemetry, Gmail and broader network sensors are separate integration paths.
- Eight agents use versioned tasks/results and evidence provenance.
- REST for operations; authorized WebSocket for live updates; durable background jobs.
- Every button has real backend behavior and complete loading/success/error/authorization states.
- Monitoring gaps are never represented as a clean security result.

Next: Database ER diagram and schema design

The next artifact should define users/account contexts, personal workspaces, organizations, organization memberships, rooms, room memberships, invitations, analyst applications/verifications, devices and ownership, data sources/Gmail connections, security events/payload references, agent tasks/results, IOC matches, correlation groups, alerts, incidents, cases, evidence, reports, payments and audit logs.

Resolve primary/foreign keys, many-to-many relationships, tenant scope, uniqueness constraints, indexes, archive/delete behavior and representative authorization queries before migrations. Then compare the ER design with existing repository models and migrations before writing code.

End of architecture blueprint